

10 - Kinematic Analysis

Regular Theo Jansen Mechanism Position Analysis

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
""" --- POSITION ANALYSIS --- """
```

```
# Initialize variables
```

```
# Actual CAD measurements of the linkages:
```

```
b = 124.50
```

```
c = 117.9
```

```
d = 120.3
```

```
e = 167.4
```

```
f = 118.2
```

```
g = 110.1
```

```
h = 197.1
```

```
i = 147
```

```
j = 150
```

```
k = 185.7
```

```
m = 45
```

```
x = 114
```

```
y = 23
```

```
"""
```

```
#Jansen Holy numbers:
```

```
x = 38 # x-component of ground link
```

```
y = 7.8 # y-component of ground link
```

```
b = 41.5 # Upper 4-bar Rocker
```

```
c = 39.3 # Lower 4-bar Rocker
```

```
d = 40.1
```

```
e = 55.8 # Rigid body "bde"
```

```
f = 39.4 # Parallel-like mechanism
```

```
g = 36.7
```

```
h = 65.7
```

```
i = 49 # Rigid body "ghi"
```

```
j = 50 # Upper 4-bar Coupler
```

```
k = 61.9 # Lower 4-bar Coupler
```

```
m = 15 # Crank
```

```
"""
```

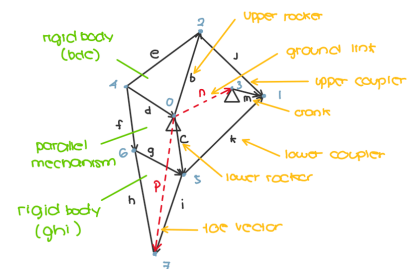


Figure 0: Jansen Mechanism

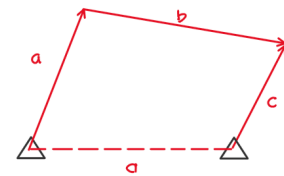


Figure 1: Reference four-bar mechanism

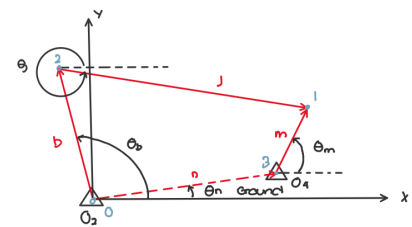


Figure 2: Vector loop (nm-jk)

```

theta_m_degrees = np.arange(360)
theta_j = []
theta_b = []
theta_k = []
theta_c = []
theta_d = []
theta_f = []
theta_g = []
theta_i = []
theta_p = []
p = []
P_p_x = []
P_p_y = []
X3 = []
Y3 = []
n = np.sqrt(x**2 + y**2) # Ground link

```

"""

In this mechanism we refer to FIGURE 0 for the links and linkages of the Theo Jansen Mechanism.

The Jansen mechanism is composed of

- crank (m)
- 2 oscillating rockers (b,c)
- 2 couplers (j, k)
- 2 three-bar linkages (b,d,e) and (g,h,i)
- upper four bar linkages (nm-bj)
- lower four bar linkages (nm-ck)
- open four-bar linkage/parallel link linkage (dc-fg)
- rigid three-bar (g,h,i)
- Ground (O2-O4)
- Toe (P)

We let O2 be the origin of the General Coordinate System.

Note that in general four bar notation we use FIGURE 1 as a reference for our analysis for the four bar linkages.

For Figure 1, we can write the vector loop equation as

$$d - c - a - b = 0$$

$$d \exp(j\theta_d) - c \exp(j\theta_c) - a \exp(j\theta_a) - b \exp(j\theta_b) = 0$$

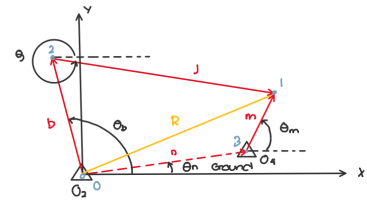


Figure 3: Vector loop (nm-jk) with R

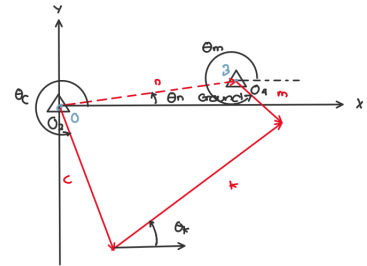


Figure 4: Vector loop (nm-ck)

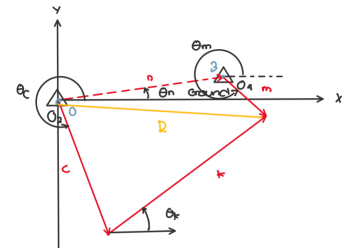


Figure 5: Vector loop (nm-ck) with R

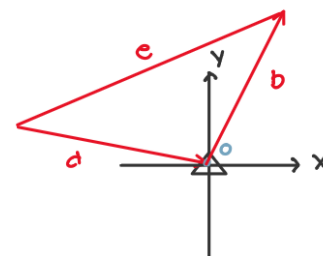


Figure 6: Triangle bde

Using these we can analyze the four linkages (nm-bj), (nm-ck).

```
"""
```

```
for angle in theta_m_degrees:
    # Compute the position of point B
    X_m = x + m * np.cos(np.deg2rad(angle)) #x comp of point B from
    origin
    Y_m = y + m * np.sin(np.deg2rad(angle)) #y comp of point B from
    origin
    R = np.sqrt(X_m**2 + Y_m**2) #vector of Point B from origin
    alpha = np.rad2deg(np.arctan2(Y_m, X_m)) #angle of point 1 wrt O2
    ground/origin
```

```
"""
```

---- VECTOR LOOP (1) UPPER 4-BAR LINKAGE (nm-bj) ---

Referring to FIGURE 2, we can write the equation of the components of the linkages as:

$$\begin{aligned} n \cos(\theta_n) + m \cos(\theta_m) - b \cos(\theta_b) - j \cos(\theta_j) &= 0 \\ n \sin(\theta_n) + m \sin(\theta_m) - b \sin(\theta_b) - j \sin(\theta_j) &= 0 \end{aligned}$$

Although a four bar linkage we use R and cos law to get θ_j and θ_b .

This is done by making a triangle for links {b,j,R}.

Let A_j be the projection of j on R.

Let A_b be the projection of b on R.

See FIGURE 3.

```
"""
```

```
A_j = (j**2 - b**2 + R**2) / (2 * j) #projecton of j on R
A_b = (b**2 - j**2 + R**2) / (2 * b) #projection of b on R
theta_j.append(np.rad2deg(np.arccos(A_j / R)) + alpha) #theta_j as
function of m
theta_b.append(np.rad2deg(np.arccos(A_b / R)) + alpha) #tehta_b as
function of m
```

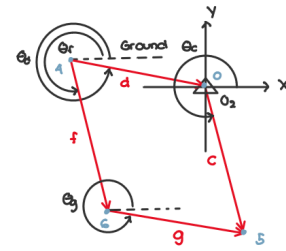


Figure 7: Parallel-link linkage vector loop (dc-fg)

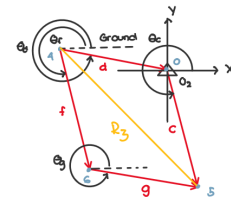


Figure 8: Parallel-link linkage vector loop (dc-fg) with R

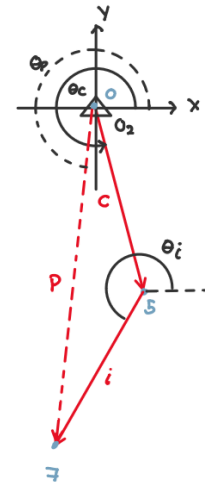


Figure 9: Vector loop (cip)

```
"""
```

```
--- VECTOR LOOP (2) LOWER 4-BAR LINAKG (nm-ck) ---
```

Again, we use R and cos law to get theta_k and theta_c.

Note that we are using the same R here since it is the same points for both (nm-bj) and (nm-ck).

This is done by making a triangle for links {c,k,R}.

Let A_k be the projecton of k on R.

Let A_c be the projection of c on R.

See FIGURE 4 and FIGURE 5.

```
"""
```

```
A_k = (k**2 - c**2 + R**2) / (2 * k) #projecton of k on R
```

```
A_c = (c**2 - k**2 + R**2) / (2 * c) #projecton of c on R
```

```
theta_k.append(np.rad2deg(np.arccos(A_k / R)) + alpha) #theta_k as  
function of m
```

```
theta_c.append(np.rad2deg(np.arccos(A_c / R)) + alpha) #theta_c as  
function of m
```

```
"""
```

```
--- PARALLEL LINKAGE (dc-fg) --
```

Need to find theta_d since it is the common link between triangle bde and parallel dc-fg.

Usining cosine law with bde FIGURE 6:

```
"""
```

```
theta_d_value = np.degrees(np.arccos((b**2 + d**2 - e**2) / (2 * b *  
d))) + theta_b[-1]
```

```
theta_d.append(theta_d_value)
```

```
"""
```

Now looking at the parallelogram (FIGURE 7), we take theta d as the new "moving ground".

We take point 4 as the input since the rotation from m is transmitted to that point.

Let Point 5 be the output since that point helps determine point P.

Just like in our analysis with the 2 four bar linkages where we used R to extend a line

from the origin/input to the output point to help with the analysis, see FIGURE 8.

We use R_3 to extend a line from point 4 to point 5.

We let X_3 and Y_3 be the components of R_3 , and α_3 be its angle from horizontal.

```
"""

X3_value = d * np.cos(np.radians(theta_d_value)) + c *
np.cos(np.radians(theta_c[-1]))
Y3_value = d * np.sin(np.radians(theta_d_value)) + c *
np.sin(np.radians(theta_c[-1]))

X3.append(X3_value)
Y3.append(Y3_value)

R3 = np.sqrt(X3[-1]**2 + Y3[-1]**2)
alpha3 = np.rad2deg(np.arctan(Y3_value/X3_value))

"""
```

Now that we have α_3 of point 5, we can use it to find θ_f and θ_g in the parallelogram.

Just as before, we use projections to help us.

Let A_f be the projection of f on R_3 .

Let A_g be the projection of g on R_3 .

We use R_3 since this is the third four-bar we are analyzing,
and the first two four-bars used the same R .

```
"""

A_f = (f**2 - g**2 + R3**2) / (2 * f) #projection of f on R3
A_g = (g**2 - f**2 + R3**2) / (2 * g) #projection of g on R3
A_f_clipped = np.clip(A_f / R3, -1, 1) #clipped to take away rounding
errors
A_g_clipped = np.clip(A_g / R3, -1, 1) #clipped to take away rounding
errors

theta_f_value = np.rad2deg(np.arccos(A_f_clipped)) + alpha3
theta_g_value = np.rad2deg(np.arccos(A_g_clipped)) + alpha3

theta_f.append(theta_f_value)
theta_g.append(theta_g_value)
```

```
"""
```

```
--- (ghi) TRIANLGE + (cip) TRIANGLE ---
```

Referring to Figure 9:

Now we need find find point p as a function of the crank angle theta_m

First we find theta i from cosine law of triangle {ghi}

```
"""
```

```
theta_i_value = theta_g[-1] - np.rad2deg(np.arccos((g**2 + i**2 -  
h**2) / (2 * g * i)))  
theta_i.append(theta_i_value)
```

```
"""
```

From the diagram, P is just the sum of the x and y components of link c and link i.

Therefore, since we already knowt theta_c from the second four-bar analysis and we obtained

theta i from the ghi triangle, we can find theta p.

```
"""
```

Angular Position

```
theta_p_value = np.rad2deg(np.arctan(  
(c * np.sin(np.deg2rad(theta_c[-1])) + i *  
np.sin(np.deg2rad(theta_i[-1]))) /  
(c * np.cos(np.deg2rad(theta_c[-1])) + i *  
np.cos(np.deg2rad(theta_i[-1])))  
))  
  
theta_p.append(theta_p_value)  
  
p_x = c * np.cos(np.deg2rad(theta_c[-1])) + i *  
np.cos(np.deg2rad(theta_i[-1]))  
p_y = c * np.sin(np.deg2rad(theta_c[-1])) + i *  
np.sin(np.deg2rad(theta_i[-1]))  
  
p_value = np.sqrt(p_x**2 + p_y**2)  
  
p.append(p_value)
```

Linear position

P_p_y.append(p_y) **# y-component of P**

P_p_x.append(p_x) **# x-component of P**

```
# Position Plot
```

```
plt.figure(1)
```

```
# Angular Position of Toe Point (P) of Walking Mechanism
```

```
plt.subplot(3, 1, 1)
```

```
plt.plot(theta_m_degrees, theta_p) # Plot against theta_m_degrees
```

```
plt.title("Angular Position of Toe Point (P) of Walking Mechanism")
```

```
plt.xlabel(r'Crank angle(deg) or  $\theta_m(^{\circ})$ ')
```

```
plt.xlim([0, 360])
```

```
plt.ylabel(r'Angular Position (deg) or  $\theta_P(^{\circ})$ ')
```

```
plt.grid(True)
```

```
# Position of Toe Point (P) of Walking Mechanism (x-direction)
```

```
plt.subplot(3, 1, 2)
```

```
plt.plot(theta_m_degrees, P_p_x) # Plot against theta_m_degrees
```

```
plt.title("Position of Toe Point (P) of Walking Mechanism (x-direction)")
```

```
plt.xlabel(r'Crank angle(deg) or  $\theta_m(^{\circ})$ ')
```

```
plt.xlim([0, 360])
```

```
plt.ylabel('Toe Point Position (mm)')
```

```
plt.grid(True)
```

```
# Position of Toe Point (P) of Walking Mechanism (y-direction)
```

```
plt.subplot(3, 1, 3)
```

```
plt.plot(theta_m_degrees, P_p_y) # Plot against theta_m_degrees
```

```
plt.title("Position of Toe Point (P) of Walking Mechanism (y-direction)")
```

```
plt.xlabel(r'Crank angle(deg) or  $\theta_m(^{\circ})$ ')
```

```
plt.xlim([0, 360])
```

```
plt.ylabel('Toe Point Position (mm)')
```

```
plt.grid(True)
```

```
# Save the figure
```

```
plt.tight_layout() # Adjust subplots to fit into figure area.
```

```
# For toe path
```

```
plt.figure(2)
```

```
plt.plot(P_p_x, P_p_y)
```

```
plt.title("Trace Path of Toe Point (P) of Walking Mechanism")
```

```
plt.xlabel('Toe Point Position (x-direction)(mm)')
```

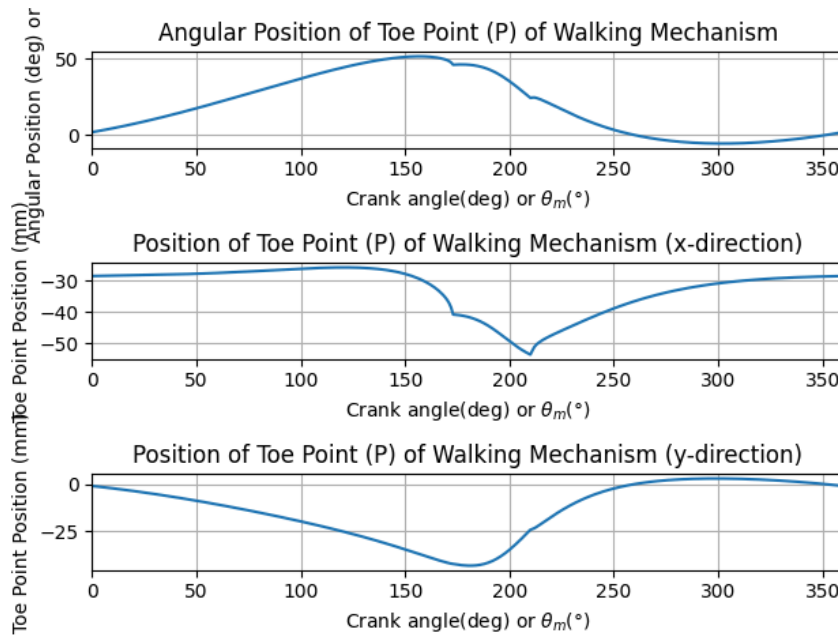
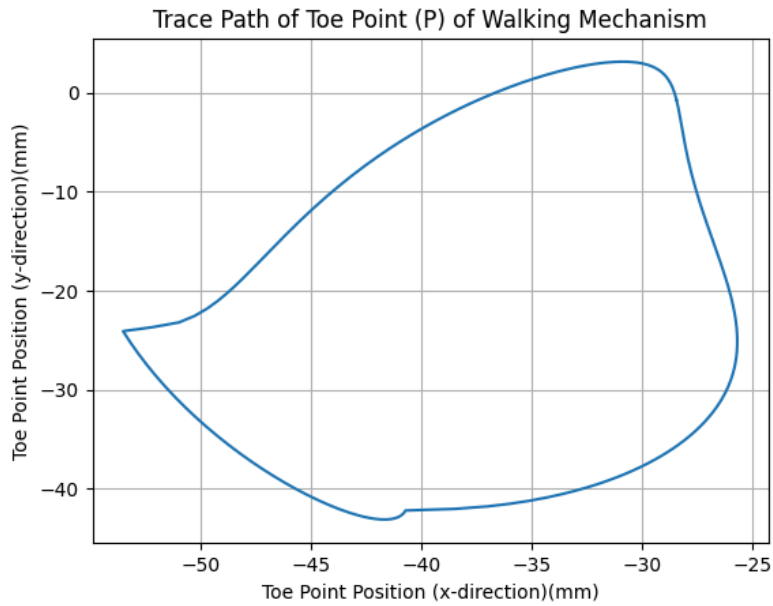
```
plt.ylabel('Toe Point Position (y-direction)(mm)')
```

```
plt.grid(True)
```

```
# Show the plots
```

```
plt.show()
```

Below is the output for the toe point position's path throughout one cycle. There are some discrepancies in certain points of the plot. This is largely due to Python's way of handling angles. The next step is to create a kinematic analysis for the proposed parallelogram mechanism to change the path of the leg to enable stair climbing. Given that Python has some discrepancies, we are moving to do that analysis in MATLAB. We intend to focus on the position analysis as this is the most important aspect of our project.



Reference: Mohsenizadeh, M., & Zhou, J. (2015). *Kinematic analysis and simulation of Theo Jansen mechanism* (Doctoral dissertation, Lamar University).